4. OPERAŢII ARITMETICE ÎN CALCULATOARE NUMERICE

Obiective:

Obiectivele principale ale acestui capitol sunt orientate spre prezentarea operaţiilor elementare de adunare, scădere, înmulţire şi împărţire, în virgulă fixă şi virgulă mobilă. Se prezintă algoritmi specifici codurilor de reprezentare direct, invers şi complementar. În acest capitol se prezintă metodele cele mai eficiente pentru efectuarea operaţiilor de înmulţire şi împărţire.

Cuprins:

4. OPERAŢII ARITMETICE ÎN CALCULATOARE NUMERICE 1

4.1 Operaţii aritmetice în virgulă fixă 2
4.1.1 Adunarea şi scăderea în virgulă fixă 2
4.1.1.1 Adunarea şi scăderea în cod direct 2
4.1.1.1.1 Adunarea în cod direct 2
4.1.1.1.2 Scăderea în cod direct 2
4.1.1.2 Adunarea şi scăderea în cod invers 3
4.1.1.2.1 Adunarea în cod invers 3
4.1.1.2.2 Scăderea în cod invers 4
4.1.1.3 Adunarea şi scăderea în cod complementar 5
4.1.1.3.1 Adunarea în cod complementar 5
4.1.1.3.2 Scăderea în cod complementar. 6
4.1.2 Înmulţirea în virgulă fixă 8
4.1.2.1 Înmultirea directă 8
4.1.2.2 Înmulţirea prin metoda BOOTH 12
4.1.2.3 Înmulţirea în complement faţă de 2 18
4.1.3 Împărţirea în virgulă fixă 23
4.1.3.1 Împărţirea în virgulă fixă prin metoda comparaţiei 23
4.1.3.2 Metoda de împărţire cu refacerea restului parţial 27
4.1.3.3 Metoda de împărţire fără refacerea restului parţial 31

4.2 Operaţii aritmetice în virgulă mobilă 35
4.2.1 Adunarea şi scăderea în virgulă mobilă 35
4.2.2 Înmulţirea în virgulă mobilă 38

4.1 Operaţii aritmetice în virgulă fixă

4.1.1 Adunarea şi scăderea în virgulă fixă

4.1.1.1 Adunarea şi scăderea în cod direct

4.1.1.1.1 Adunarea în cod direct

Fie :

$x = x_s x_{-1} x_{-2} \ldots x_{-n}$

$y = y_s y_{-1} y_{-2} \ldots y_{-n}$

cu condiţia ca $|x| + |y| < 1$

două numere în virgulă fixă, subunitare, reprezentate în cod direct.

Operaţia de adunare se realizează prin metoda comparaţiei:

dacă $x_s = y_s$ *atunci* $x + y = x_s(|x|+|y|)$

altfel *dacă* $|x| < |y|$ *atunci* $x + y = y_s(|y|-|x|)$

altfel $x + y = y_s(|x|-|y|)$

Se observă că pentru a realiza operaţia de adunare în cod direct a unor numere reprezentate în virgulă fixă este necesar un dispozitiv de comparare a modulelor celor două numere.

Operaţia de adunare constă în:

Dacă semnele celor doi operanzi sunt identice atunci se adună modulele celor două numere şi se stabileşte semnul comun;

Dacă semnele celor doi operanzi sunt diferite atunci se compară modulele celor două numere şi în funcţie de relaţia dintre ele se scade cel mai mic din cel mai mare şi se dă semnul numărului cel mai mare în valoare absolută.

Dispozitivul de adunare trebuie să dipună şi de o unitate aritmetică logică, care să poată efectua atât operaţia de adunare cât şi de scădere. Din punctul de vedere al implementării fizice un astfel de dispozitiv este destul de complex.

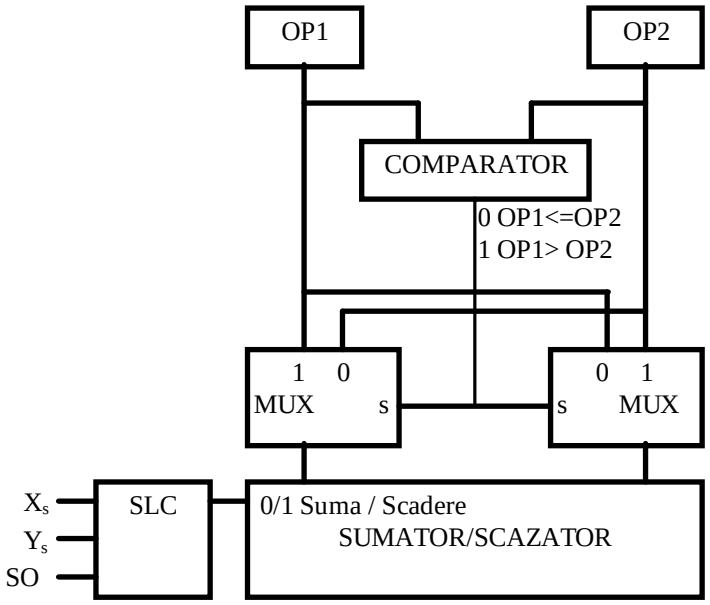
4.1.1.1.2 Scăderea în cod direct

Operaţia de scădere în cod direct se reduce la adunare, după ce în prealabil s-a negat bitul de semn al scăzătorului.

Operaţia propriu-zisă efectuată de dispozitivul aritmetic este funcţie de semnul celor doi operanzi şi de operaţia cerută.

Astfel, *dacă* x_s , y_s sunt semnele celor doi operanzi, iar
s_o semnul operaţiei dorite (0 adunare, 1 scădere) şi
s_{of} semnul operaţiei efectuate fizice
atunci $s_{of} = x_s \oplus y_s \oplus s_o$

Schema bloc de principiu a unui dispozitiv care efectuează operaţii de adunare şi scădere în cod direct este prezentată în Fig. 4.1.



Unitate aritmetică pentru adunare/scadere în cod direct

Figura 4.1

4.1.1.2 Adunarea şi scăderea în cod invers

4.1.1.2.1 Adunarea în cod invers

Fie :
$x = x_s x_{-1} x_{-2} \ldots x_{-n}$
$y = y_s y_{-1} y_{-2} \ldots y_{-n}$
cu condiţia ca $|x| + |y| < 1$

două numere în virgulă fixă , subunitare, reprezentate în cod invers.

Operaţia de adunare se efectuează astfel: se adună cei doi operanzi bit cu bit începând cu rangul cel mai putin semnificativ, inclusiv bitul de semn. Eventualul transport din bitul de semn se adună la cifra cea mai puţin semnificativă a rezultatului.

Justificare:

1. în cazul în care $x>0$, $y>0$ şi $|x|+|y|<1$

$[x]_i + [y]_i = |x|+|y| = [|x|+|y|]_i$

2. în cazul în care $x>0$, $y<0$

cazul 1: $|x|>|y|$

$[x]_i + [y]_i = |x|+2-|y|-2^{-n} =2 + (|x|-|y|) - 2^{-n}$

Deoarece $|x|-|y|>0$ atunci $2+(|x|-|y|)$ generează un transport din bitul de semn, care conform regulii enunţate se adună la poziţia cea mai putin semnificativă a rezultatului, adică transportul devine 2^{-n}.

În acest caz $[x]_i + [y]_i = (|x|-|y|) = [(|x|-|y|)]_i$

cazul 2: $|x|<|y|$

$[x]_i + [y]_i = |x|+2-|y|-2^{-n} =2 - (|y|-|x|) - 2^{-n}$

deoarece $|y|-|x|>0$,

atunci $2-(|y|-|x|)$ nu generează un transport din bitul de semn.

În acest caz $[x]_i + [y]_i =2 - (|y|-|x|) - 2^{-n} = [-(|y|-|x|)]_i$

3. în cazul în care $x<0$, $y<0$ şi $|x|+|y|<1$

$[x]_i + [y]_i = 2 - |x| -2^{-n} + 2 - |y| -2^{-n} =$

$= 2 + 2 - (|x|+|y|) -2^{-n} -2^{-n} =$

$= 2 - (|x|+|y|) -2^{-n} = [-(|x|+|y|)]_i$

deoarece $(|x|+|y|)>0$,

atunci $2 - (|x|+|y|)$ nu generează un transport din bitul de semn.

În schimb $2 + 2 - (|x|+|y|)$ generează un transport care prin adunare la poziţia cea mai putin semnificativă devine 2^{-n}.

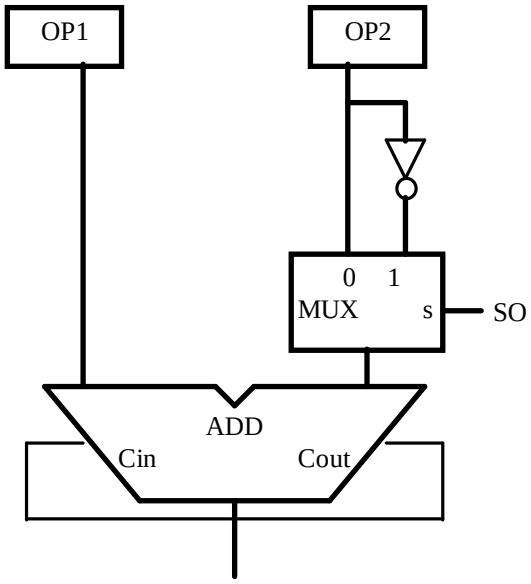
În acest caz $[x]_i + [y]_i = [-(|x|+|y|)]_i$

4.1.1.2.2 Scăderea în cod invers

Operaţia de scădere în cod invers se reduce la adunare după ce în prealabil s-a inversat (complementat faţă de 1) scăzătorul.

Schema bloc de principiu a unui dispozitiv de adunare şi scădere în cod invers este prezentată în Fig. 4.2. Se observă că este mult mai simplu decât cel pentru reprezentarea prin mărime şi semn. În plus bitul de semn participă la fel ca ceilalţi biţi care specifică mărimea.

Acest dispozitiv are un dezavantaj că necesită adunarea eventualului transport din bitul de semn ceea ce conduce la un timp de propagare mare. În cazuri extreme unitatea de adunare, scădere în cod invers poate fi lentă datorită propagării transportului.



Unitatea aritmetică logică pentru adunare/scădere în cod invers

Figura 4.2

4.1.1.3 Adunarea şi scăderea în cod complementar

4.1.1.3.1 Adunarea în cod complementar

Fie :

$x = x_s x_{-1} x_{-2} \ldots x_{-n}$

$y = y_s y_{-1} y_{-2} \ldots y_{-n}$

cu condiţia ca $|x| + |y| < 1$

două numere în virgulă fixă, subunitare, reprezentate în cod complementar.

Operaţia de adunare constă în: se adună bit cu bit începând cu rangul cel mai putin semnificativ, inclusiv bitul de semn. Eventualul transport din bitul de semn se neglijează.

Justificare:

1. în cazul în care x>0, y>0 şi |x|+|y|<1
$[x]_c + [y]_c = |x|+|y| = [(|x|+|y|)]_c$

2. în cazul în care x>0, y<0
cazul 1: |x|>|y|
$[x]_c + [y]_c = |x|+2-|y| = 2 + (|x|-|y|)$
deoarece |x|-|y|>0,
atunci 2+(|x|-|y|) generează un transport din bitul de semn, care conform regulii enunţate se neglijează.
În acest caz:
$[x]_c + [y]_c = (|x|-|y|) = [(|x|-|y|)]_c$

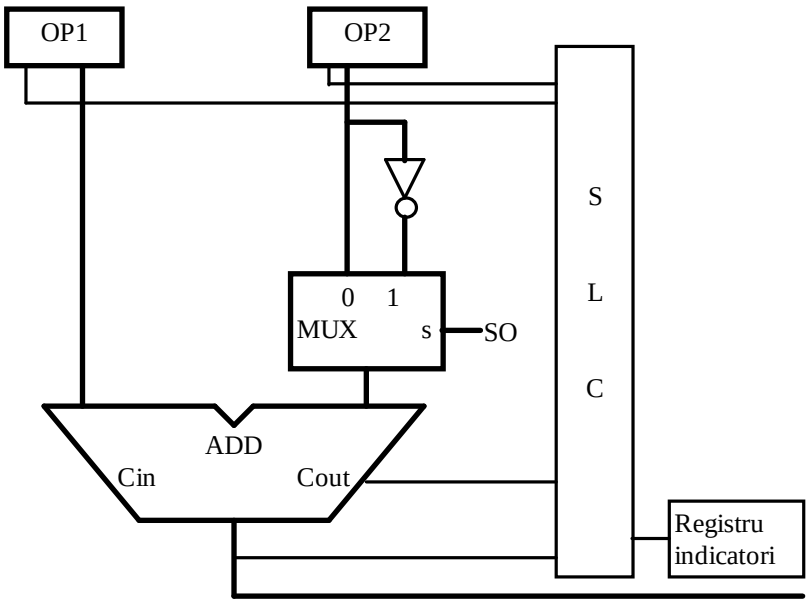
cazul 2: |x|<|y|
$[x]_c + [y]_c = |x|+2-|y| = 2 - (|y|-|x|)$
deoarece |y|-|x|>0,
atunci 2-(|y|-|x|) nu generează un transport din bitul de semn.
În acest caz:
$[x]_c + [y]_c = 2 - (|y|-|x|) = [-(|y|-|x|)]_c$

3. în cazul în care x<0, y<0 şi |x|+|y|<1
$[x]_c + [y]_c = 2 - |x| + 2 - |y| =$
$= 2 + 2 - (|x|+|y|) =$
$= 2 - (|x|+|y|) = [-(|x|+|y|)]_c$
Deoarece (|x|+|y|)>0,
atunci 2 - (|x|+|y|) nu generează un transport din bitul de semn.
În schimb 2 + 2 - (|x|+|y|) generează un transport care se neglijează.
În acest caz:
$[x]_c + [y]_c = [-(|x|+|y|)]_c$

4.1.1.3.2 Scăderea în cod complementar.

Operaţia de scădere în cod complementar se reduce la adunare după ce în prealabil s-a complementat scăzătorul.

Schema bloc de principiu a unui dispozitiv de adunare şi scădere în cod complementar



Unitate aritmetică pentru adunare/scădere în cod complementar

Figura 4.3

este prezentată în Fig. 4.3. Se observă că este mult mai simplu decât cel pentru reprezentarea prin mărime şi semn. În plus bitul de semn participă la fel ca ceilalţi biţi care specifica mărimea.

Indicatorii de condiţii asociaţi unităţii de adunare scădere.

Vom prezenta indicatorii de condiţii numai pentru unitatea care lucrează cu numere reprezentate în complement faţă de doi.

Indicatorii propuşi sunt următorii:

S semn, reprezintă valoarea bitului de semn al rezultatului.

Z zero, este poziţionat în unu dacă rezulatul este zero, şi este poziţionat în zero când rezultatul este diferit de zero.

Valoarea lui Z se stabileşte astfel: $Z = \vee / \overline{\text{REZULTAT}}$

D depăşire, este poziţionat în unu, când cele două numere care se adună sunt pozitive şi există transport spre bitul de semn, sau dacă operanzii sunt negativi şi nu există transport spre bitul de semn.

Considerând x_s, y_s semnele celor doi operanzi şi t_s transportul spre bitul de semn, care atunci când apare modifică pe z_s, semnul rezultatului astfel că valoarea lui D se stabileşte cu ecuaţia:

$$D = \bar{x}_s \wedge \bar{y}_s \wedge z_s \vee x_s \wedge y_s \wedge \bar{z}_s$$

T transport, se poziţionează pe 1 în cazul în care există un transport din bitul de semn spre stânga.

P paritate, se poziţionează pe 1 în cazul în care rezulatul are un număr par de unităţi.

4.1.2 Înmulţirea în virgulă fixă

Pentru realizarea operaţiei de înmulţire cele mai utilizate metode se bazează pe adunarea repetată. Există o serie de algoritmi care au la bază adunarea repetată dintre care cei mai cunoscuţi sunt:

- înmulţirea directă;
- metoda von Newmann;
- metodele lui Robertson;
- metoda Booth;
- înmulţirea în complement faţă de 2;
- metoda înmulţirii scurte;
- înmulţirea rapidă cu salvarea transportului;

În cele ce urmează nu vom trata toate metodele ci vom prezenta numai algoritmii pentru: înmulţirea directă, metoda Booth, înmulţirea în complement faţă de 2.

4.1.2.1 Înmulţirea directă

Metoda de înmulţire directă este utilizată în cazul reprezentării în cod direct (mărime şi semn).

Fie:

$x = x_s x_{-1} x_{-2} \ldots x_{-n}$

$y = y_s y_{-1} y_{-2} \ldots y_{-n}$

două numere în virgulă fixă, subunitare, reprezentate în cod direct.

Pentru a realiza înmulţirea **z = x * y** este necesar ca rezultatul să fie reprezentat pe un număr dublu de biţi sau să se realizeze o rotunjire a rezultatului, în cazul în care se reprezintă pe acelaşi număr de biţi.

Operaţia de înmulţire constă din trei etape principale:

a) - determinarea semnului;
b)- înmulţirea efectivă;
c)- rotunjirea rezultatului;

a) **Determinarea semnului**

$\mathbf{z_s = x_s \oplus y_s}$

b) **înmulţirea efectivă**

Înmulţirea efectivă are ca scop determinarea modulului produsului: $|z| = |x| * |y|$

$$|z| = y_{-1}*2^{-1}|x| + y_{-2}*2^{-2}|x| + ... + y_{-n}*2^{-n}|x|$$

Din punct de vedere hardware nu este eficient să se realizeze suma produselor parţiale conform relaţiei de mai sus deoarece sunt necesare **n** registre care să păstreze produsele parţiale şi un sumator cu **n** termeni de lungime 2*n biţi.

Calcularea modulului rezultatului se poate face printr-un proces iterativ de calcul al produselor parţiale de forma:

$p_{-n} = 0 + |x| * y_{-n}$
$p_{-n+1} = p_{-n} *2^{-1} + |x| * y_{-n+1}$
$p_{-n+2} = p_{-n+1} *2^{-1} + |x| * y_{-n+2}$
....
$p_{-n+j} = p_{-n+j-1} *2^{-1} + |x| * y_{-n+j}$
....
$p_{-1} = p_{-2} *2^{-1} + |x| * y_{-1}$

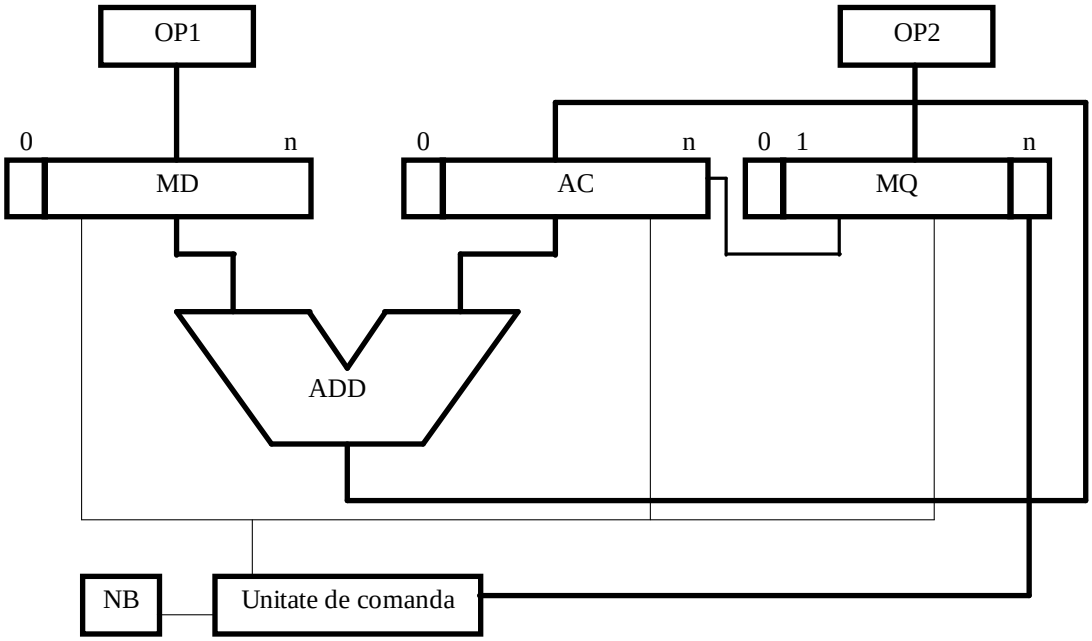
Se observă că după ce a participat la formarea produsului parţial cifra y_{-j} nu mai este necesară şi deci produsul parţial se poate forma într-un registru acumulator care se deplasează la dreapta cu o poziţie, bitul cel mai putin semnificativ putând fi preluat în registrul ce păstrează înmulţitorul y.

$$|x| * |y| = p_{-1} * 2^{-1}$$

c) **Rotunjirea rezultatului**

Dacă |z| trebuie să fie depus într-un cuvânt de n+1 biţi, atunci trebuie făcută o rotunjire prin adaus ce constă în a aduna 2^{-n} la rezultat dacă partea la care se renunţă are bitul cel mai semnificativ 1.

Pentru realizarea unui dispozitiv aritmetic paralel care efectuează înmulţirea a doi operanzi reprezentaţi în virgulă fixă, cod direct se propune schema bloc din Fig. 4.4.



Unitate aritmetică pentru înmulţire în cod direct

Figura 4.4

unde:
MD registrul ce păstrează primul operand (OP1)
MQ registrul ce păstrează al doilea operand (OP2)
AC registrul ACUMULATOR ce păstrează produsul parţial
NB registrul ce contorizează numărul de biţi
Rezultatul se obţine în : AC,MQ

Algoritmul de înmulţire în cod direct este următorul:

1. se citeşte deânmulţitul în registrul MD;
   se citeşte înmulţitorul în registrul MQ;
   se iniţializează registrul acumulator AC ← 0;
   se iniţializează registrul contor de biţi NB ← 0;

2. *dacă* MD=0 sau MQ=0
   atunci rezultat 0 , gata operaţie;

3. *atât timp cât* NB < n
   dacă $MQ_n = 1$ *atunci* AC ← AC + MD;

   deplasează dreapta cu o poziţie AC, MQ;
   incrementează NB;

4. stabilire semn :$AC_s = MD_s \oplus MQ_s$;

Descrierea AHPL a unităţii de comandă a dispozitivului de înmulţire în cod direct este:

MODULE : Dispozitiv de înmulţire cod direct;
MEMORY : MD[n+1]; AC[n+1]; MQ[n+1]; NB[m] /* $2^m \geq n$ */
INPUTS : OP1[n+1]; OP2[n+1]
OUTPUTS : AC[n+1]; MQ[n+1]

/* se încarcă operanzii
se iniţializează AC şi NB
1. MD ← OP1; MQ ← OP2; AC ← (n+1)T0; NB ← mT0

/* se verifică dacă unul din operanzi este egal cu 0
2. → $((v/MD_{1:n}) \wedge (v/MQ_{1:n}))/(4)$

/* dacă da se iniţializează MQ şi gata operaţie
3. $MQ_{1:n}$ ← (nT0)
→ (7)

/* se adună deînmulţitul la produsul parţial, dacă bitul curent analizat este 1
/* se realizează deplasarea prin conexiuni decalate la dreapta cu o poziţie (de la unitatea aritmetică logică la registrul AC) şi se incrementează contorul de biţi.

4. $AC,MQ \leftarrow ((0,ADD(AC;MD;0),MQ_{1:n-1})!(0,AC,MQ_{1:n-1}))*(MQ_n,\overline{MQ_n})$
NB ← INC(NB);

/* se verifică dacă s-au analizat toţi biţii
/* dacă nu, se reia operaţia
5. → $v/(NB \oplus (mTn))$ / (4)

/* se stabileşte semnul rezultatului
6. $AC_0 \leftarrow MD_0 \oplus MQ_0$

7. DEAD END

END SEQUENCE

END

Exemplu:
Să considerăm:
MD = -5/16 deînmulţitul reprezentat în cod direct pe 5 biţi
= 10101
MQ = -6/16 înmulţitorul reprezentat în cod direct pe 5 biţi

NB	MD	AC	MQ	Observaţii:
	1		1	bitul de semn nu participă
000	0101	00000	011<u>0</u>	0 deplasare AC,MQ
001	0101	00000 <u>00101</u> 00101	001<u>1</u>	1 adunare AC ← AC + MD deplasare AC,MQ
010	0101	00010 <u>00101</u> 00111	100<u>1</u>	1 adunare AC ← AC + MD deplasare AC,MQ
011	0101	00011	110<u>0</u>	0 deplasare AC,MQ
100	0101	00001	1110	stabilire semn $AC_0=MD_0 \oplus MQ_0$
		00001	1110	Rezultatul se obţine în AC,MQ AC,MQ = 000011110= 30/256

Dacă se doreşte o reprezentare numai pe 5 biţi, atunci se face o rotunjire prin adaus
AC=00010, AC = 2/16

4.1.2.2 Înmulţirea prin metoda BOOTH

Pentru a realiza operaţia de înmulţire a unor numere reprezentate în complement faţă de 2, prin adunări repetate, este necesar să evaluăm înmulţitorul. Evaluarea înmulţitorului reprezentat în complement faţă de 2 se realizează utilizând formula lui Booth.

Dacă

$y = y_s y_{-1} y_{-2} ... y_{-n}$ este un număr în complement faţă de 2 valoarea reprezentată de acest număr este:

$$V_y = -y_s + \sum_{i=1}^{n} y_{-i} * 2^{-i}$$ formula lui Booth

Justificare:

a) dacă y este pozitiv $y = 0y_{-1}y_{-2}...y_{-n}$ are valoarea

$$V_y = 0 + \sum_{i=1}^{n} y_{-i} * 2^{-i}$$

ceea ce reprezintă într-adevăr evaluarea unui număr pozitiv reprezentat în cod direct.

b) dacă y este negativ, $y = 1y_{-1}y_{-2}...y_{-n}$, valoarea sa este obţinută evaluând $[-y]_c$.

Dar ştim că $[y]_c = [y]_i + 2^{-n}$, deci

$$V_{[-y]c} = \bar{y}_{-1}\bar{y}_{-2}...\bar{y}_{-n} + 2^{-n} \text{ unde } \bar{y}_{-j} = 1 - y_{-j}$$

$$V_{[-y]c} = \left(\sum_{i=1}^{n} \bar{y}_{-i} * 2^{-i}\right) + 2^{-n}$$

$$V_{[-y]c} = \left(\sum_{i=1}^{n} (1 - y_{-i} * 2^{-i})\right) + 2^{-n}$$

$$V_{[-y]c} = \sum_{i=1}^{n} 2^{-i} - \left(\sum_{i=1}^{n} y_{-i} * 2^{-i}\right) + 2^{-n}$$

$$V_{[-y]c} = 1 - \sum_{i=1}^{n} y_{-i} * 2^{-i}$$

Dar $V_y = -V_{[-y]c}$ deci $V_{[-y]c} = -1 + \sum_{i=1}^{n} y_{-i} * 2^{-i}$ ceea ce

justifică formula lui Booth.

Metoda de înmulţire Booth este adecvată pentru înmulţirea numerelor reprezentate în virgulă fixă, cod complementar.

Fie :

$x = x_s x_{-1} x_{-2} \ldots x_{-n}$

$y = y_s y_{-1} y_{-2} \ldots y_{-n}$

două numere în virgulă fixă, subunitare, reprezentate în cod complementar.

Pentru a realiza $z = x * y$ este acelaşi lucru cu a realiza $z = x * V_y$

Booth a propus ca o cifrele de reprezentare ale înmulţitorului, în cod complementar, să se înlocuiască cu diferenţa a două cifre adiacente:

$y_{-j} \leftarrow y_{-j-1} - y_{-j} \quad 1 \le j \le n$

unde y_{-n-1} se consideră 0.

În acest caz:

$z = x * ((y_{-1}-y_s)+(y_{-2}-y_{-1})*2^{-1}+(y_{-3}-y_{-2})*2^{-2}+\ldots+(y_{-n-1}-y_{-n})*2^{-n})$

$z = x * (-y_s + y_{-1}*2^{-1} + y_{-2}*2^{-2} +\ldots+y_{-n}*2^{-n})$

$z = x * V_y$

$z = x * y$

ceea ce ne arată că înlocuirea propusă de Booth conduce de altfel la realizarea directă a produsului, unde numerele sunt reprezentate în cod complementar.

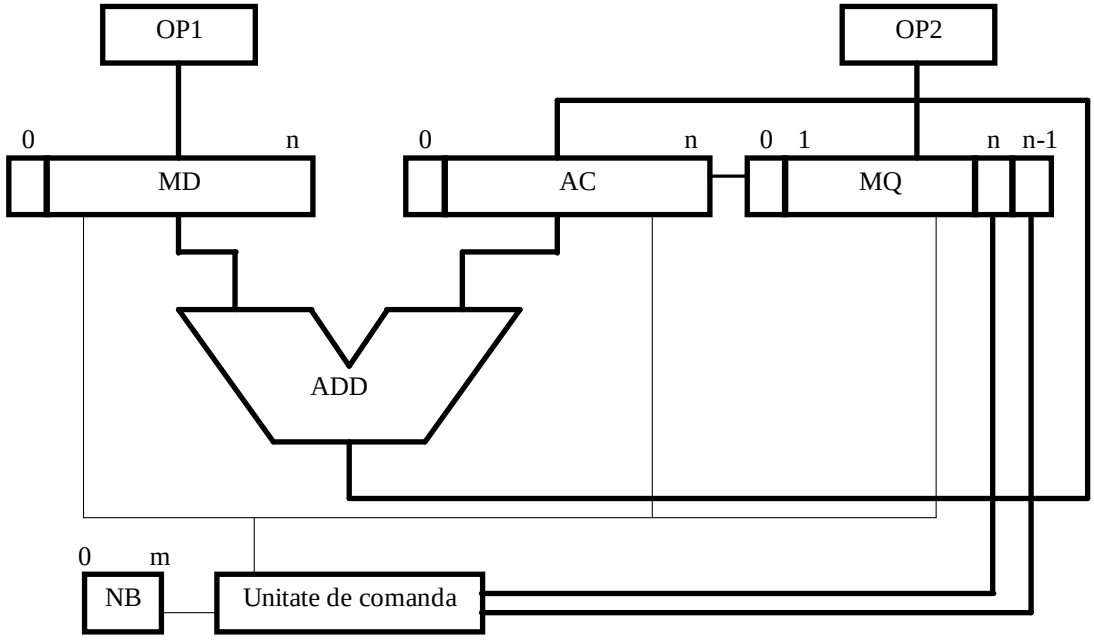
Înlocuirea propusă de Booth nu se realizează efectiv, ci conduce numai la analiza a doi biţi adiacenţi din reprezentarea înmulţitorului (nu a unui singur bit cum se realiza la înmulţirea directă).

Astfel dacă

y_{-j}	y_{-j-1}	
0	1	se deplasează produsul parţial cu o poziţie la dreapta;
0	1	se adună deînmulţitul la produsul parţial şi apoi se deplasează rezultatul cu o poziţie la dreapta;
1	0	se scade deînmulţitul din produsul parţial şi apoi se deplasează rezultatul cu o poziţie la dreapta;
1	1	se deplasează produsul parţial cu o poziţie la dreapta;

După ultima operaţie, în care participă şi bitul de semn, nu se mai efectuează operaţia de deplasare.

Pentru realizarea unui dispozitiv aritmetic paralel care efectuează înmulţirea a doi operanzi reprezentaţi în virgulă fixă, cod complementar prin metoda Booth se propune schema bloc din Fig. 4.5.



Unitate aritmetică pentru înmulţire prin metoda Booth

Figura 4.5

unde:

MD registrul ce păstrează primul operand (OP1)
MQ registrul ce păstrează al doilea operand (OP2)
AC registrul ACUMULATOR ce păstrează produsul parţial
NB registrul ce contorizează numărul de biţi
Rezultatul se obţine în : AC,MQ

Algoritmul de înmulţire prin metoda Booth este următorul:

1. se citeşte deânmulţitul în registrul MD;
   se citeşte înmulţitorul în registrul MQ;
   se iniţializează registrul acumulator AC ← 0;
   se iniţializează registrul contor de biţi NB ← 0;

se iniţializează bitul $MQ_{n+1} \leftarrow 0$;

2. *dacă* MD=0 sau MQ=0
   atunci rezultat 0 , gata operaţie;

3. *atât timp cât* NB < n + 1
   dacă MQ_n,MQ_{n+1} = 01 *atunci* AC ← AC + MD;
   dacă MQ_n,MQ_{n+1} = 10 *atunci* AC ← AC - MD;

   deplasează dreapta cu o poziţie AC,MQ;
   incrementează NB;

4. după ultima operaţie nu este necesară deplasarea

Descrierea AHPL a unităţii de comandă a dispozitivului de înmulţire în cod direct este:

MODULE : Dispozitiv de înmulţire metoda Booth;
MEMORY : MD[n+1]; AC[n+1]; MQ[n+2]; NB[m] /* $2^m \geq n+1$ */
INPUTS : OP1[n+1]; OP2[n+1]
OUTPUTS : AC[n+1]; MQ[n+1]

/* se încarcă operanzii şi se iniţializează AC şi NB
1. MD ← OP1; MQ ← OP2,0; AC ← (n+1)T0; NB ← mT0

/* se verifică dacă un operand este egal cu 0
2. → $((\vee/MD) \wedge (\vee/MQ))/(4)$

/* dacă da atunci se iniţializează MQ şi gata operaţie
3. MQ ← (n+1)T0
   → (9)

/* se analizează doi biţi adiacenţi curenţi dacă sunt 00 sau 11 se face numai deplasare mai puţin la ultima operaţie (la care a participat şi bitul de semn) cînd nu se mai face deplasarea */

4. → $\overline{MQ_n \oplus MQ_{n+1}}$ /(6)

/* dacă biţii adiacenţi sunt 10 sau 01 se realizează scădere respectiv adunare şi deplasare */

5. $AC \leftarrow (ADD(AC;\overline{MD};1) \; ! \; (ADD(AC;MD;0)) * (MQ_n,MQ_{n+1})$
/* se verifică dacă s-au analizat toţi biţii

6. $\rightarrow (v/(NB \oplus mT(n+1))/(9)$

/* dacă nu s-au analizat toţi biţii se incrementează contorul de biţi
7. $NB \leftarrow INC(NB)$

/* se realizează deplasarea produsului parţial împreună cu înmulţitorul
8. $AC,MQ \leftarrow AC_0,AC,MQ_{0:n-1}$
$\rightarrow (4)$

9. DEAD END

END SEQUENCE

END

Exemplu:
Să considerăm operanzii :
MD = -5/16 reprezentat în cod complementar pe 5 biţi 11011
MQ = -6/16 reprezentat în cod complementar pe 5 biţi 11010

NB	MD	AC	MQ	Observaţii:
000	11011	00000	110100	starea iniţială
				00 deplasare AC,MQ
001	11011	00000	011010	10 scădere AC←AC-MD
		00101	011010	
		00101		deplasare AC,MQ
010	11011	00010	101101	01 adunare AC ← AC+MD
		11011		
		11101		deplasare AC,MQ
011	11011	11110	110110	10 scădere AC ← AC-MD
		00101		
		00011		deplasare AC,MQ
100	11011	00001	111011	11 deplasare AC,MQ
				dar fiind ultima nu se efectuează.
				Rezultatul se obţine în AC,MQ
				AC,MQ = 000011110
				AC,MQ = 30/256

4.1.2.3 Înmulţirea în complement faţă de 2

În cazul numerelor reprezentate în cod complementar utilizarea înmulţirii directe necesită cicli suplimentari pentru procesul de complementare.

Fie :

$x = x_s x_{-1} x_{-2} \ldots x_{-n}$

$y = y_s y_{-1} y_{-2} \ldots y_{-n}$

două numere în virgulă fixă, subunitare, reprezentate în cod complementar.

Pentru realizarea înmulţirii trebuie ca deînmulţitul să fie adunat cu el însuşi de un număr de ori dat de valoarea lui y. Practic trebuie calculat |y| şi după aceea trebuie adunat sau scăzut de |y| ori deânmulţitul cu el însuşi în funcţie de semnul lui y, dacă este pozitiv sau negativ.

Pentru a obţine |y| trebuie să complementăm pe y dacă acesta este negativ şi să-l lăsăm nemodificat dacă acesta este pozitiv.

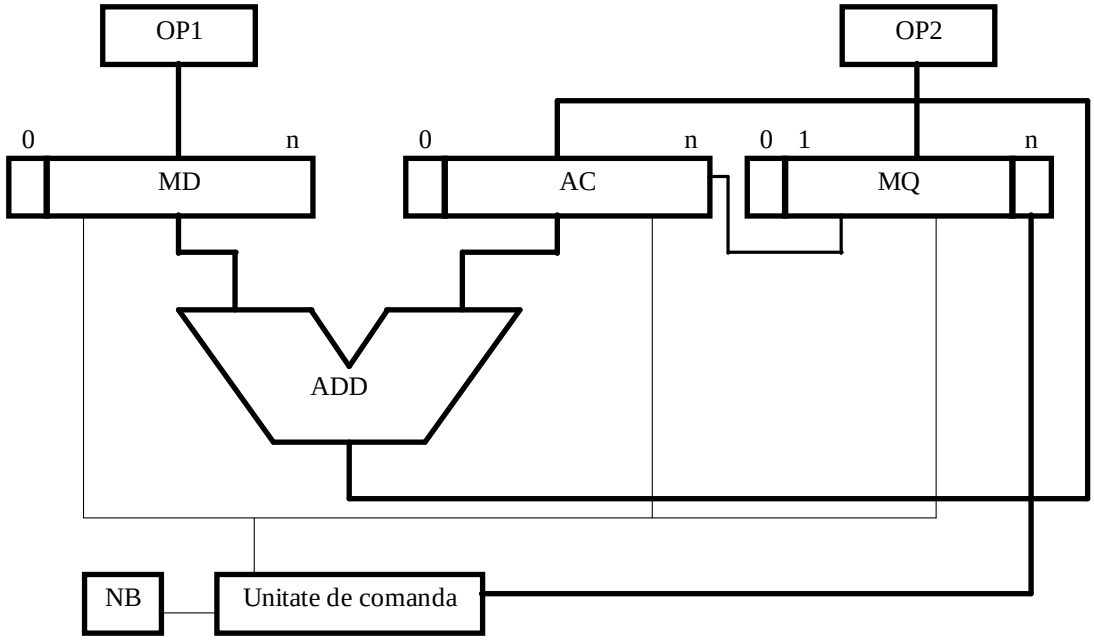
Înmulţirea în complement faţă de 2 se poate realiza în aceeaşi manieră cu înmulţirea în cod direct, numai că în loc să complementăm operanzii înainte de efectuarea operaţiei, vom complementa numai înmulţitorul simultan cu examinarea biţilor acestuia.

Realizarea complementului înmulţitorului se face pe baza regulii: se parcurg de la dreapta la stânga biţii acestuia (de la bitul cmps la bitul cms), zerourile şi prima unitate întîlnite rămân neschimbate, restul biţilor se neagă.

Amintind că la operaţia de înmulţire în cod direct analiza biţilor se făcea începând cu bitul cmps, şi dacă bitul era 1 se realiza adunarea deânmulţitului la produsul parţial şi deplasarea produsului parţial, iar dacă bitul era egal cu 0 se realiza numai deplasarea produsului parţial.

În cazul înmulţirii în cod complementar se aplică aceeaşi regulă ca la cod direct până la întâlnirea primei unităţi, după care se interpretează biţii invers ca la metoda în cod direct. În acest fel se realizează din "mers" complementarea înmulţitorului.

Pentru realizarea unui dispozitiv aritmetic paralel care efectuează înmulţirea a doi operanzi reprezentaţi în virgulă fixă, cod complementar prin metoda înmulţirii în complement faţă de 2 se propune schema bloc din Fig. 4.6.



Unitate aritmetică pentru înmulţire în cod complementar

Figura 4.6

unde:

MD registrul ce păstrează primul operand (OP1)
MQ registrul ce păstrează al doilea operand (OP2)
AC registrul ACUMULATOR ce păstrează produsul parţial

NB registrul ce contorizează numărul de biţi
Rezultatul se obţine în : AC,MQ

Algoritmul de înmulţire prin metoda complementului faţă de 2 este următorul:

1. se citeşte deânmulţitul în registrul MD;
   se citeşte înmulţitorul în registrul MQ;
   se iniţializează registrul acumulator AC ← 0;
   se iniţializează registrul contor de biţi NB ← 0;

2. *dacă* MD=0 sau MQ=0
   atunci rezultat 0 , gata operaţie;

3. *atât timp cât* MQn = 0
   deplasează aritmetic dreapta cu o poziţie MQ;
   incrementează contorul de biţi NB;
4. *atât timp cât* NB < n+1
   dacă MQ_0 = 0 *atunci* AC ← AC + MD
   altfel AC ← AC - MD
5. *dacă* MQ_0 = 0 *atunci* deplasează aritmetic dreapta AC,MQ
   cu extensie de bit MD_0
   altfel deplasează aritmetic dreapta AC,MQ
   cu extensie de bit $\overline{MD_0}$

   incrementează contor de biţi NB
/* din acest moment interpretăm invers biţii înmulţitorului dacă acesta este negativ.

6. *dacă* MQ_0 = 0 (înmulţitor pozitiv)
   atunci dacă MQ_n = 1 (bitul curent din înmulţitor)
   atunci salt la 4
   altfel salt la 5
   atunci dacă MQ_n = 0 (bitul curent din înmulţitor)
   atunci salt la 4
   altfel salt la 5

Descrierea AHPL a unităţii de comandă a dispozitivului de înmulţire în cod direct este:

MODULE : Dispozitiv de înmulţire cod direct;
MEMORY : MD[n+1]; AC[n+1]; MQ[n+1]; NB[m] /* $2^m \geq n$ */
INPUTS : OP1[n+1]; OP2[n+1]
OUTPUTS : AC[n+1]; MQ[n+1]

/* se încarcă operanzii se iniţializează AC şi NB */
1. MD ← OP1; MQ ← OP2; AC ← (n+1)T0; NB ← mT0

/* se verifică dacă un operand este egal cu zero
2. → ((v/MD) ∧ (v/MQ))/(4)

/* dacă da se iniţializează MQ şi gata operaţie
3. MQ ← (n+1)T0
→ (10)

/* se elimină zerourile cele mai puţin semnificative
4. → (MQ_n)/(6)

5. MQ ← MQ_0,$MQ_{0:n-1}$
NB ← INC(NB)
→ (4)

/* pentru primul 1 întâlnit se adună sau se scade deînmulţitul la produsul parţial în functie de semnul înmulţitorului.

6. AC ← (ADD(AC;MD;0) ! ADD(AC;$\overline{MD}$;1)) * ($\overline{MQ_0}$,MQ_0)

/* se deplasează produsul parţial, împreună cu înmulţitorul cu o poziţie la dreapta cu extensia bitului de semn al deînmulţitului (direct sau negat) în funcţie de semnul înmulţitorului se incrementează totodata şi contorul de biţi.

7. AC,$MQ_{1:n}$ ← ((MD_0 ! $\overline{MD_0}$)*(MQ_0 ! $\overline{MQ_0}$)),AC,$MQ_{1:n:1}$
NB ← INC(NB)

/* se verifică dacă s-au parcurs toţi biţi, dacă da, gata operaţie

8. $\rightarrow (v/(NB \oplus (mTn))/(10)$

/* din acest moment se realizează complementarea din "mers" interpretând invers biţii înmulţitorului numai dacă acesta este negativ.

9. $\rightarrow ((MQ_{n-1} \oplus MQ_{0)}, \overline{MQ_{n-1} \oplus MQ_0}) /(6,7)$

10.DEAD END

END SEQUENCE

END

4.1.3 Împărţirea în virgulă fixă

Operaţia de împărţire în binar se realizează după principii asemănătoare calculului manual. Împărţirea se reduce la o serie de scăderi succesive ale împărţitorului din restul parţial.

După modul în care se efectuează succesiunea operaţiilor de scădere sau alte operaţii care au în fond acelasi efect, se disting trei metode pricipale:

- metoda comparaţiei;
- metoda refacerii restului parţial (metoda regenerării);
- metoda în care nu se reface restul parţial.

Metoda aleasă depinde în primul rând de forma de reprezentare a numerelor în calculator.

Metodele comparaţiei şi regenerării se aplică numai numerelor reprezentate în **cod direct** iar ultima metodă se aplică oricărei forme de reprezentare necesitând totuşi nişte corecţii.

Pentru primele două metode dacă numerele se păstrează sub o altă formă decât cea în mărime şi semn trebuie făcută o transformare a formei de reprezentare.

În cazul împărţirii apar două probleme:

- abandonarea efectuării operaţiei de împărţire dacă împărţitorul este egal cu 0, sau dacă deâmpărţitul este mai mare decât împărţitorul, pentru reprezentare în virgulă fixă, numere subunitare sau dacă deâmpărţitul este mai mic decât împărţitorul, pentru reprezentare în virgulă fixă, numere întregi;
- rotunjirea rezultatului.

4.1.3.1 Împărţirea în virgulă fixă prin metoda comparaţiei

Metoda comparaţiei se utilizează în cazul în care operanzii sunt reprezentaţi în cod direct.

Fie :

$x = x_s x_{-1} x_{-2} \ldots x_{-n}$

$y = y_s y_{-1} y_{-2} \ldots y_{-n}$

două numere în virgulă fixă, subunitare, reprezentate în cod direct care îndeplinesc condiţiile $|x| < |y|$ şi $y \neq 0$.

Vom considera:

$x = x_s\ x_{-1}\ x_{-2} \ldots\ x_{-n}$ deâmpărţitul;

$y = y_s\ y_{-1}\ y_{-2} \ldots\ y_{-n}$ împărţitorul;

$q = q_s\ q_{-1}\ q_{-2} \ldots\ q_{-n}$ câtul;

$r_i = r_{is}r_{i-1}r_{i-2}...r_{i-n}$ al i-lea rest parţial;

$r = r_s\, r_{-1}\, r_{-2}... r_{-n}$ restul împărţirii;

Conform teoremei fundamentale a împărţirii avem:

$x = q * y + r$

unde $|x| < |y|$ şi $y \neq 0$

Etapele operaţiei de împărţire sunt:

Iniţial se presupune primul rest parţial $r_0 = |x|$.

Dacă restul parţial curent r_i este mai mare decât împărţitorul se efectuează operaţia de scădere a împărţitorului din restul parţial şi se generează ca cifră a câtului valoarea 1. În caz contrar nu se efectuează operaţia de scădere şi se generează ca cifră a câtului valoarea 0.

În ambele situaţii restul parţial se deplasează la stânga cu o poziţie şi devine noul rest parţial.

Deci:

$r_0 = |x|$

$r_1 = 2 * r_0 - q_{-1} * |y|$ unde *dacă* $2*r_0 > |y|$ *atunci* $q_{-1} = 1$

altfel $q_{-1} = 0$

$r_2 = 2 * r_1 - q_{-2} * |y|$ unde *dacă* $2*r_1 > |y|$ *atunci* $q_{-2} = 1$

... *altfel* $q_{-2} = 0$

...

$r_i = 2 * r_{i-1} - q_{-i} * |y|$ unde *dacă* $2*r_{i-1} > |y|$ *atunci* $q_{-i} = 1$

altfel $q_{-i} = 0$

Ţinând seama că :

$$2^{-i} * r_i = 2^{-i+1} * r_{i-1} - 2^{-i} * q_{-i} * |y|$$

Înlocuînd prima relaţie în a doua, a doua în a treia, etc şi înmulţind cu 2^{-n} ambii termeni ai expresiei finale se obţine:

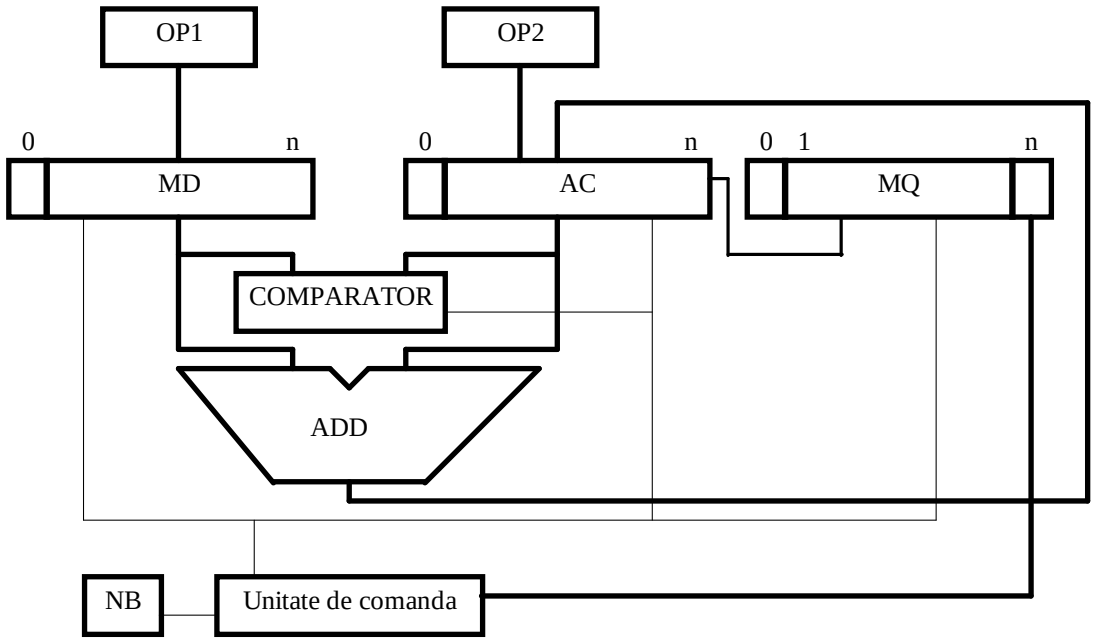
$$2^{-n}*r_n = |x| - 2^{-1}*q_{-1}*|y| - 2^{-2}*q_{-2}*|y| - ... -2^{-n}*q_{-n}*|y|$$

$$|x| / |y| = |q| + 2^{-n}* r_n / |y|$$

Restul împărţirii este $r = 2^{-n}*r_n$

Operaţia de împărţire se termină în cazul în care restul parţial curent $r_i = 0$ sau când s-a calculat un număr de biţi ai câtului egal cu lungimea cuvântului calculator în care se reprezintă operanzii.

Pentru realizarea unui dispozitiv aritmetic paralel care efectuează împărţirea a doi operanzi reprezentaţi în virgulă fixă, cod direct prin metoda comparaţiei se propune schema bloc din Fig.4.7.



Unitate aritmetică pentru împărţire prin metoda comparaţiei

Figura 4.7

unde:

MD registrul ce păstrează împărţitorul
MQ registrul ce păstrează câtul
AC registrul ACUMULATOR ce păstrează iniţial deîmpărţitul şi restul parţial curent
NB registrul ce contorizează numărul de biţi
Rezultatul se obţine în : MQ câtul
AC restul

Algoritmul de împărţire prin metoda comparaţiei este următorul:

1. se citeşte împărţitorul în registrul MD;
   se citeşte deîmpărţitul în registrul AC;
   se încarcă eventual registrul MQ cu extensia deîmpărţitului dacă acesta este pe lungime dublă;

se iniţializează registrul contor de biţi NB ← 0;

bitul de semn al operanzilor nu participă decât la stabilirea semnului

2. *dacă* MD = 0
atunci împărţire prin zero, gata operaţie.
dacă AC ≥ MD
atunci depăşire, gata operaţie
dacă AC = 0 şi MQ ≤ MD
atunci rezultat zero, gata operaţie.

3. deplasează AC,MQ la stînga cu o poziţie cu introducere $MQ_n \leftarrow 0$
AC,MQ ← 2(AC,MQ),0

4. *atât timp cât* NB < n
dacă AC ≥ MD
atunci AC ← AC - MD
AC,MQ ← 2(AC,MQ),1 deplasează cu introducere 1
altfel AC,MQ ← 2(AC,MQ),0 deplasează cu introducere 0
incrementează contorul de biţi NB ← INC(NB)

5. stabileşte semn rezultat $MQ_s = AC_s \oplus MD_s$
Se obţine în registrul MQ câtul împărţirii iar în registrul AC restul.

Exemplu:

Să considerăm:
AC = -5/16 deâmpărţitul reprezentat în cod direct pe 5 biţi
= 10101
MD = -6/16 împărţitorul reprezentat în cod direct pe 5 biţi
= 10110

NB	MD	AC	MQ	Observaţii:
	1	1	0	bitul de semn nu participă
000	0110	0101	0000	$r_0 = \|x\|$
		1010	0000	$2r_0$ $(2r_0 > \|y\|)$
		0110		realizăm scăderea
001		0100	0001	$r_1 = 2r_0 - \|y\|$; $q_{-1}=1$
		1000	0010	$2r_1$ $(2r_1 > \|y\|)$
		0110		realizăm scăderea
010		0010	0011	$r_2 = 2r_1 - \|y\|$; $q_{-2}=1$
		0100	0110	$2r_2$ $(2r_2 < \|y\|)$
011		0100	0110	$r_3 = 2r_2$ $q_{-3}=0$
		1000	1100	$2r_3$ $(2r_3 > \|y\|)$
		0110		realizăm scăderea
100		0010	1101	$r_4 = 2r_3 - \|y\|$; $q_{-4}=1$
				Semn $MQ_s = 1 \oplus 1 = 0$
				Câtul MQ = 01101
				Restul AC = 0010 (restul se consideră înmulţit cu 2^{-n})

Verificare:
AC/MD = 01101 rest 00000010

Se observă că se respectă relaţia fundamentală a împărţirii:
|x| = |y| * |q| + r
0.0101 = 0.0110 * 0.1101 + 0.00000010

Pentru realizarea dispozitivului de împărţire prin metoda comparaţiei trebuie să existe un circuit de comparare. Pentru a evita necesitatea acestui circuit de comparare se poate realiza compararea prin scădere. În acest fel se ajunge la metoda refacerii restului parţial.

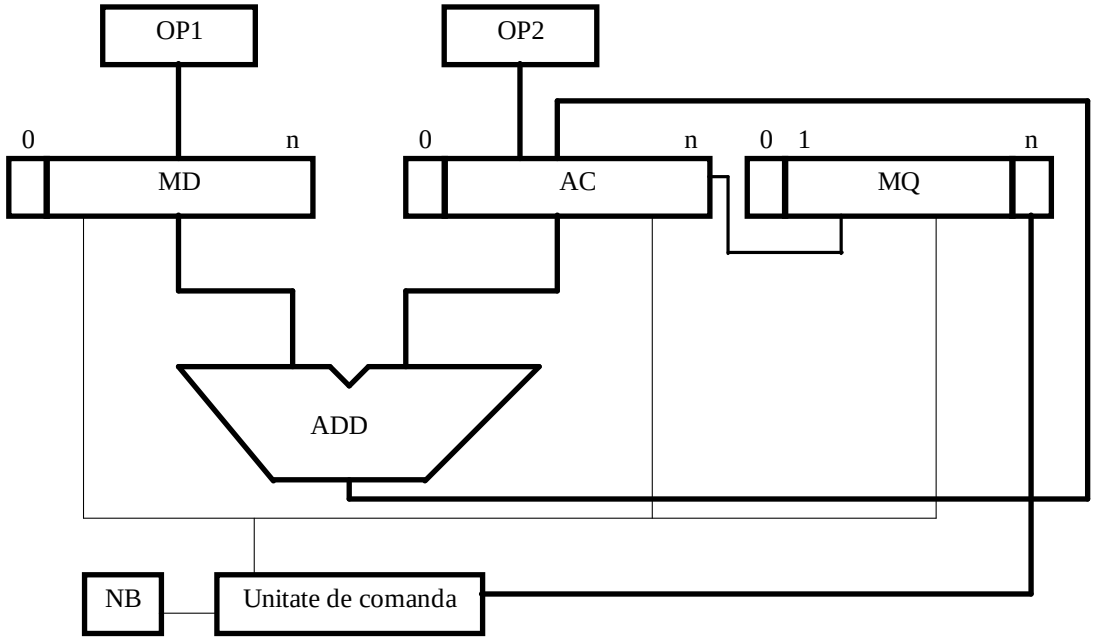
4.1.3.2 Metoda de împărţire cu refacerea restului parţial

Această metodă poate fi considerată ca un caz special al metodei comparaţiei. Compararea se poate face prin scădere.

Dacă diferenţa este pozitivă (echivalent cu faptul că restul parţial este mai mare decât împărţitorul) se obţine drept cifră a câtului 1 şi procesul continuă cu o deplasare la stânga cu o poziţie.

Dacă diferenţa este negativă (echivalent cu faptul că restul parţial este mai mic decât împărţitorul) se obţine drept cifră a câtului 0. Deoarece în acest caz nu trebuia să scădem împărţitorul din restul parţial, trebuie să procedăm la refacerea restului parţial prin adunarea împărţitorului la acesta.

Pentru realizarea unui dispozitiv aritmetic paralel care efectuează împărţirea a doi operanzi reprezentaţi în virgulă fixă, cod direct prin metoda refacerii restului parţial se propune schema bloc din Fig. 4.8, asemănătoare cu cea folosită pentru metoda comparaţiei în care nu mai există comparatorul.



Unitate aritmetică pentru împărţire prin metoda refacerii

Figura 4.8

unde:

MD registrul ce păstrează împărţitorul

MQ registrul ce păstrează câtul

AC registrul ACUMULATOR ce păstrează iniţial deâmpărţitul şi restul parţial curent

NB registrul ce contorizează numărul de biţi

Rezultatul se obţine în : MQ câtul

AC restul

Algoritmul de împărţire prin metoda refacerii restului parţial este următorul:

1. se citeşte împărţitorul în registrul MD;
   se citeşte deâmpărţitul în registrul AC;
   se încarcă eventual registrul MQ cu extensia deîmpărţitului dacă acesta este pe lungime dublă;
   se iniţializează registrul contor de biţi NB ← 0;

bitul de semn al operanzilor nu participă decât la stabilirea semnului

2. *dacă* MD = 0
   atunci împărţire prin zero, gata operaţie.
   dacă AC ≥ MD
   atunci depăşire, gata operaţie
   dacă AC = 0 şi MQ ≤ MD
   atunci rezultat zero, gata operaţie.

3. deplasează AC,MQ la stânga cu o poziţie cu introducere $MQ_n \leftarrow 0$
   AC,MQ ← 2(AC,MQ),0

4. *atât timp cât* NB < n
   AC ← AC - MD
   dacă rezultat pozitiv
   atunci AC,MQ ← 2(AC,MQ),1 deplasează cu introducere 1
   altfel AC ← AC + MD **reface restul parţial**
   AC,MQ ← 2(AC,MQ),0 deplasează cu introducere 0
   incrementează contorul de biţi NB ← INC(NB)

5. stabileşte semn rezultat $MQ_s = AC_s \oplus MD_s$

Se obţine în registrul MQ câtul împărţirii iar în registrul AC restul.

Exemplu:
Să considerăm:
AC = -5/16 deâmpărţitul reprezentat în cod direct pe 5 biţi
= 10101

MD = -6/16 împărţitorul reprezentat în cod direct pe 5 biţi
= 10110

NB	MD	AC	MQ	Observaţii:
	1	1	0	bitul de semn nu participă
000	0110	0101	0000	$r_0 = \|x\|$
		1010		$2r_0$
		0110		realizăm scăderea
001		0100	0001	$r_1 = 2r_0 - \|y\|$
				rezultat pozitiv ; $q_{-1}=1$
		1000	0010	$2r_1$
		0110		realizăm scăderea
010		0010	0011	$r_2 = 2r_1 - \|y\|$
				rezultat pozitiv ; $q_{-2}=1$
		0100	0110	$2r_2$
		0110		realizăm scăderea
		10010		$r_3 = 2r_2 - \|y\|$
				rezultat negativ
		0110 +		refacem restul parţial
011		0100	0110	$q_{-3}=0$
		1000	1100	$2r_3$
		0110		realizăm scăderea
100		0010	1101	$r_4 = 2r_3 - \|y\|$; $q_{-4}=1$
				Semn $MQ_s = 1 \oplus 1 = 0$
				Câtul MQ = 01101
				Restul AC = 0010 (restul se consideră înmulţit cu 2^{-n})

Verificare:
AC/MD = 01101 rest 00000010

Se observă că se respecta relaţia fundamentală a împărţirii:
|x| = |y| * |q| + r
0.0101 = 0.0110 * 0.1101 + 0.00000010

Metoda împărţirii prin refacerea restului parţial elimină comparatorul, în schimb necesită un timp destul de lung datorită procesului de refacere a restului parţial.

4.1.3.3 Metoda de împărţire fără refacerea restului parţial

Analizând metoda refacerii restului parţial se observă că fiecare operaţie de refacere a restului parţial este urmată de o scădere a împărţitorului în ciclul următor.

Ultimele două operaţii : adunarea împărţitorului (operaţia de refacere) şi scăderea (la ciclul următor) împărţitorului din restul parţial deplasat la stânga cu o poziţie, pot fi combinate într-o singură operaţie de adunare a împărţitorului la ciclul următor (în loc de scădere)

$2*(r_i + |y|) - |y|$ echivalent cu $2*r_i + |y|$

Metoda de împărţire în care restul parţial nu se reface constă în a scădea sau aduna împărţitorul din restul parţial curent în funcţie de semnele împărţitorului şi restului parţial.

Dacă aceste semne sunt identice se efectuează o scădere şi se introduce drept cifră a câtului valoarea 1, iar dacă semnele sunt diferite se face o adunare şi se ia ca cifră a câtului valoarea 0.

Algoritmul de împărţire prin metoda refacerii restului parţial este următorul:

1. se citeşte împărţitorul în registrul MD;
   se citeşte deîmpărţitul în registrul AC;
   se încarcă eventual registrul MQ cu extensia deâmpărţitului dacă acesta este pe lungime dublă;
   se iniţializează registrul contor de biţi NB $\leftarrow$ 0;

2. *dacă* MD = 0
   atunci împărţire prin zero, gata operaţie.
   dacă AC $\geq$ MD
   atunci depăşire, gata operaţie
   dacă AC = 0 şi MQ $\leq$ MD
   atunci rezultat zero, gata operaţie.

3. deplasează AC,MQ la stânga cu o poziţie cu introducere $MQ_n \leftarrow 0$
   AC,MQ $\leftarrow$ 2(AC,MQ),0

4. *atât timp cât* NB < n
   dacă $AC_s \oplus MD_s = 0$

atunci AC ← AC - MD

AC,MQ ← 2(AC,MQ),1 deplasează cu introducere 1

altfel AC ← AC + MD

AC,MQ ← 2(AC,MQ),0 deplasează cu introducere 0

incrementează contorul de biţi NB ← INC(NB)

5. Dacă operanzii sunt reprezentaţi în alt cod decât cod direct se face o corecţie stabileşte semn rezultat $MQ_s = AC_s \oplus MD_s$

Se obţine în registrul MQ câtul împărţirii iar în registrul AC restul.

Exemplu:

Să considerăm:

AC = 5/16 deîmpărţitul reprezentat în cod direct pe 5 biţi
= 00101

MD = 6/16 împărţitorul reprezentat în cod direct pe 5 biţi
= 00110

NB	MD	AC	MQ	Observaţii:
000	00110	00101	0000	$r_0 = \|x\|$
		01010		$2r_0$
		00110		realizăm scăderea
001		00100	0001	$r_1 = 2r_0 - \|y\|$
				$AC_s = MD_s$; $q_{-1}=1$
		01000	0010	$2r_1$
		00110		realizăm scăderea
010		00010	0011	$r_2 = 2r_1 - \|y\|$
				$AC_s = MD_s$; $q_{-2}=1$
		00100	0110	$2r_2$
		00110		realizăm scăderea
		10010		$r_3 = 2r_2 - \|y\|$
				$AC_s \neq MD_s$
011		10010	0110	$q_{-3}=0$
		10100	1100	$2r_3$
		00110		realizăm adunarea
100		00010	1101	$r_4 = 2r_3 + \|y\|$; $q_{-4}=1$
				Semn $MQ_s = 1 \oplus 1 = 0$
				Câtul MQ = 01101
				Restul AC = 0010 (restul se consideră înmulţit cu 2^{-n})

Verificare:
AC/MD = 01101 rest 00000010
Se observă că se respectă relaţia fundamentală a împărţirii:
|x| = |y| * |q| + r
0.0101 = 0.0110 * 0.1101 + 0.00000010

Exemplu:
Să considerăm:
AC = -5/16 deîmpărţitul reprezentat în cod complementar pe 5 biţi
= 11011
MD = -6/16 împărţitorul reprezentat în cod complementar pe 5 biţi
= 11010

NB	MD	AC	MQ	Observaţii:
000	11010	11011	0000	$r_0 = \|x\|$
		10110		$2r_0$
		00110		realizăm scăderea(+ c'2 MD)
001		11100	0001	$r_1 = 2r_0 - \|y\|$
				$AC_s = MD_s$; $q_{-1}=1$
		11000	0010	$2r_1$
		00110		realizăm scăderea(+ c'2 MD)
010		11110	0011	$r_2 = 2r_1 - \|y\|$
				$AC_s = MD_s$; $q_{-2}=1$
		11100	0110	$2r_2$
		00110		realizăm scăderea(+ c'2 MD)
		00010		$r_3 = 2r_2 - \|y\|$
				$AC_s \neq MD_s$
011		00010	0110	$q_{-3}=0$
		00100	1100	$2r_3$
		11010		realizăm adunarea
100		11110	1101	$r_4 = 2r_3 + \|y\|$; $q_{-4}=1$
				$AC_s = Md_s$
				Semn $MQ_s = 1 \oplus 1 = 0$
				Câtul MQ = 01101
				Restul AC = 11110
				(restul se consideră înmulţit cu 2^{-n})

Verificare:

AC/MD = 01101 rest 11111110

Se observă că se respectă relaţia fundamentală a împărţirii:

|x| = |y| * |q| + r

1.1011 = 1.1010 * 0.1101 + 1.11111110

În exemplele arătate s-a considerat efectuarea efectivă a operaţiei de scădere, adunare specificând numai rezultatul.

În cazul în care biţii reprezentării participă direct la operaţia de înmulţire prin metoda fără refacerea restului parţial este necesară o corecţie care constă în a aduna $-1 + 2^{-n}$.

4.2 Operaţii aritmetice în virgulă mobilă

4.2.1 Adunarea şi scăderea în virgulă mobilă

Fie o reprezentare în virgulă mobilă :

$x = x_{sm}c_1c_2...c_km_{-1}m_{-2}...m_{-n}$

unde

x_{sm} reprezintă semnul mantisei

$C = c_1c_2...c_k$ reprezintă caracteristica

$M = m_{-1}m_{-2}...m_{-n}$ reprezintă mantisa

Caracteristica corespunde reprezentării cu exponent deplasat, C = E + depls. în cazul de faţă depls = 2^{k-1}

Caracteristica este întotdeauna pozitivă $C \geq 0$ iar mantisa este normalizată, $1/b \leq M \leq 1$, unde b este baza de reprezentare.

În general b=2 sau b=16. Valoarea reprezentată de număr este $V_x = (-1)^{xsm} * M * b^E$

Utilizarea reprezentării cu caracteristică în loc de reprezentarea cu exponent (deşi calculul valorii numărului se face în funcţie de exponent), se face deoarece introduce avantaje cum ar fi:

- nu este necesar un bit explicit de semn pentru exponent;
- circuitele de comparare utilizate pentru reprezentarea în virgulă fixă se utilizează şi în virgulă mobilă;
- valoarea zero se reprezintă cu 0 în toţi biţii de reprezentare ai numărului, ceea ce constituie o posibilitate de verificare.

Pentru operaţiile în virgulă mobilă considerăm operanzii:

$x = x_sx_cx_m$ primul operand

$y = y_sy_cy_m$ al doilea operand

şi

$z = z_sz_cz_m$ rezultatul

unde am notat simplificat :

x_s semnul mantisei lui x

x_c caracteristica lui x

x_m mantisa lui x

Operaţiile de adunare şi scădere a mantiselor se pot efectua numai dacă caracteristicile sunt egale.

Etapele algoritmului de adunare în virgulă mobilă sunt:

1. **egalizarea caracteristicilor** (dacă este posibil) ce constă în compararea caracteristicilor celor doi operanzi x, y şi deplasarea la dreapta a mantisei operandului cu exponentul mai mic, de un număr de ori egal cu diferenţa dintre caracteristici. Dacă diferenţa dintre caracteristici este mai mare decât numărul de biţi ai mantisei, rezultatul adunării este egal cu numărul cel mai mare (deoarece prin deplasarea la dreapta a mantisei de un număr de ori mai mare decât numărul de biţi alocaţi reprezentării acesteia, mantisa devine 0).

2. **adunarea mantiselor** după regulile cunoscute.

3. **analiza rezultatului** şi efectuarea unor operaţii intermediare.

- dacă în urmă adunarii mantiselor ($x_s = y_s$) se obţine depăşire, mantisa trebuie deplasată la dreapta cu o poziţie cu preluarea depăşirii, şi caracteristica incrementată (pentru ca rezultatul să nu se modifice). Este posibil ca în încercarea de incrementare a caracteristicii să depăşim valoarea maximă a acesteia. În acest caz se poziţionează un indicator de stare eof $\leftarrow$ 1 , exponent overflow, ceea ce indică faptul că a existat o depăşire superioară şi rezultatul nu se poate reprezenta în calculator.
- Dacă în urma adunării mantiselor ($x_s \neq y_s$) se obţine un rezultat nenormalizat ($z_m < 1/b$) se realizează normalizarea mantisei ce constă în deplasarea acesteia cu o poziţie la stînga şi decrementarea caracteristicii. Operaţiile de deplasare şi decrementare continuă pînă cînd $M \geq 1/b$. În timpul efectuării normalizarii, se verifică dacă caracteristica a ajuns la valoarea minimă şi nu mai poate fi decrementată. În caz de depăşire inferioară, se poziţionează un indicator de stare **euf $\leftarrow$ 1**, exponent underflow, ceea ce reprezintă că numărul este foarte mic şi poate fi aproximat cu 0.

Algoritmul de realizare a operaţiei de adunare a doi operanzi reprezentaţi în virgulă mobilă este următorul:

Fie :

$x = x_s x_c x_m$ primul operand

$y = y_s y_c y_m$ al doilea operand

1. încarcă primul operand în registrul A
   încarcă al doilea operand în registrul B

2. *dacă* $A_c > B_c$
   atunci dacă $A_c - B_c >$ nr. biţi mantisa
   atunci C $\leftarrow$ A gata operaţie.
   altfel deplasează B_m de A_c-B_c ori
   $C_c \leftarrow A_c$

altfel dacă $A_c < B_c$
atunci dacă $B_c - A_c >$ nr. biţi mantisa
atunci $C \leftarrow B$ gata operaţie.
altfel deplasează A_m de B_c-A_c ori
$C_c \leftarrow B_c$
altfel $C_c \leftarrow B_c$

3. $C_m \leftarrow A_m + B_m$

4. *dacă* $A_s = B_s$
atunci dacă există depăşire la C_m
atunci dacă $C_c \geq C_{max}$
atunci **eof ← 1**, gata operaţie
altfel deplasează dreapta C_m cu prel dep.
incrementează C_c
stabileşte semn rezultat
gata operaţie.
altfel dacă $C_m = 0$
atunci $C_s C_c C_m \leftarrow 0$, zero curat în toţi biţii.
gata operaţie
altfel atât timp cât $C_m < 1/b$, C_m nenormalizată
dacă $C_c \leq 0$
atunci **euf ← 0**, gata operaţie
deplasează stânga C_m
decrementează C_c
stabileşte semn rezultat
gata operaţie.

Pentru efectuarea operaţiei de scădere în virgulă mobilă, algoritmul este asemănător cu cel al adunării cu deosebirea că se modifică două decizii. Una când B_c-A_c>nr. biţi mantisă, rezultatul este $C \leftarrow -B$ şi se modifică ramurile deciziei când A_s=B_s.

Algoritmul pentru operaţia de scădere este:

1. încarcă primul operand în registrul A
încarcă al doilea operand în registrul B

2. *dacă* $A_c > B_c$

atunci dacă A_c - B_c > nr. biţi mantisa
atunci C ← A gata operaţie.
altfel deplasează B_m de A_c-B_c ori
$C_c \leftarrow A_c$
altfel dacă $A_c < B_c$
atunci dacă B_c - A_c > nr. biţi mantisa
atunci C ← -B gata operaţie.
altfel deplasează A_m de B_c-A_c ori
$C_c \leftarrow B_c$
altfel $C_c \leftarrow B_c$

3. $C_m \leftarrow A_m - B_m$

4. *dacă* $A_s \neq B_s$
atunci dacă există depăşire la C_m
atunci dacă $C_c \geq C_{max}$
atunci **eof ← 1**, gata operaţie
altfel deplasează dreapta C_m cu prel dep.
incrementează C_c
stabileşte semn rezultat
gata operaţie.
altfel dacă $C_m = 0$
atunci $C_sC_cC_m \leftarrow 0$, zero curat în toţi biţii.
gata operaţie
altfel atât timp cât $C_m < 1/b$, C_m nenormalizată
dacă $C_c \leq 0$
atunci **euf ← 0**, gata operaţie
deplasează stânga C_m
decrementează C_c
stabileşte semn rezultat
gata operaţie.

4.2.2 Înmulţirea în virgulă mobilă

Operaţia de înmulţire în virgulă mobilă constă în a aduna caracteristicile celor doi operanzi şi de a înmulţi mantisele. Trebuie ţinut seama de faptul că rezultatul trebuie să fie reprezentat sub aceeaşi formă, adică exponentul rezultatului să fie deplasat iar mantisa

normalizată. Prin adunarea celor două caracteristici se obţine exponentul rezultatului adunat cu de două ori deplasamentul. De aceea trebuie făcută corecţia necesară.

Algoritmul pentru operaţia de înmulţire este:

1. încarcă primul operand în registrul A
încarcă al doilea operand în registrul B

2. *dacă* A = 0 sau B = 0
atunci C ← 0 , se stabileşte zero în toţi biţii
gata operaţie.

3. $C_c \leftarrow A_c + B_c$

4. *dacă* $C_c > C_{max}$
atunci **eof ← 1**, gata operaţie

5. *dacă* $C_c < C_{min}$
atunci **euf ← 1**, gata operaţie.

6. corectează C_c (prin scăderea unui deplasament)

7. $C_m \leftarrow A_m * B_m$ (utilizînd una din metodele prezentate)

8. *dacă* $C_m < 1/b$, C_m nenormalizat
atunci dacă $C_c \leq 0$
atunci **euf ← 1**, gata operaţie
altfel deplasează stânga C_m
decrementează C_c
altfel gata operaţie

Deoarece ambii exponenţi erau deplasaţi:
$A_c = A_e + depls$
$B_c = B_e + depls$

În urma adunării de la pasul 3. s-a obţinut:
$C_c = A_e + depls + B_e + depls = C_e + depls + depls$,
şi a fost necesară corecţia de la pasul

6. Această corecţie nu a fost realizată imediat după adunarea caracteristicilor, deoarece este mai uşor să detectăm depăşirea superioară şi inferioară dacă există de două ori deplasamentul.

În urma operaţiei de adunare a caracteristicilor se obţine depăşire superioară dacă bitul cel mai semnificativ al lui C_c este 1 şi există depăşire spre stânga.

În urma operaţiei de adunare a caracteristicilor se obţine depăşire inferioară dacă bitul cel mai semnificativ al lui C_c este 0 şi nu există depăşire spre stânga.